Language Models, Embeddings, and Representations

DSA4213 Assignment 1 | Natural Language Processing for Data Science

SAHISHNAV KRITIN KARAMARATHUR SENTHIL KUMAR | A0357345X

1. Research questions and design

This study asks three linked questions: (i) how do count-based and neural language models trade prediction quality against training and inference cost; (ii) does freezing separately trained or public word embeddings help an LSTM; and (iii) does a better next-token model provide a more useful fixed-length representation for sentiment classification? The experiments are comparisons under a limited CPU budget, not claims of architectural superiority in general.

Corpus and preprocessing

Parts I and II use the official WikiText-2 raw splits (Salesforce/wikitext; wikitext-2-raw-v1). Empty rows were removed; text was lowercased and split on whitespace, preserving punctuation and WikiText markup. A 10,000-entry vocabulary was constructed from training text only; rare or unseen words map to <unk> and <pad> is reserved for padding. The train/validation/test splits contain 2,051,910 / 213,886 / 241,211 tokens. Unknown-token rates are 8.02% / 9.75% / 10.64%, respectively.

Component	Shared or model-specific settings
Data and selection	Official splits; training-only vocabulary; seed 42; best validation checkpoint
Neural training	32-token independent chunks; batch 32; Adam 0.001; five epochs; gradient clip 1.0
RNN / LSTM	100-dimensional trainable token embeddings; 1 layer; hidden size 128
Transformer	100-dimensional token and positional embeddings; 2 causal layers; 4 heads; FF 256; GELU; dropout 0.1
Trigram	Add-alpha unigram smoothing (alpha=0.1); weights (0.2, 0.4, 0.4) selected on validation

Fair comparison and evaluation scope

All neural models predict the next token from preceding tokens, with padding ignored in cross-entropy. Each independently processed 32-token input chunk supplies 32 shifted targets. To match these targets, the trigram was reevaluated within chunk boundaries: it uses a unigram/bigram mixture for the first target and a trigram mixture thereafter. Thus target positions are identical, although the first-position prediction mechanisms differ. We report held-out average cross-entropy and perplexity, exp(cross-entropy).

Model size, training time and inference throughput are reported alongside predictive quality. Training times include validation in each neural epoch; trigram time measures count construction. Throughput is teacher-forced rather than autoregressive generation. These differences are explicit because a numerical comparison without an aligned workload can be misleading.

Computational environment

All runs used a Windows laptop with Qualcomm Snapdragon X X1-26100 (ARM64 Oryon, 8 physical/logical cores, 2.96 GHz base), 16 GB RAM, Python 3.12.0 and PyTorch 2.14.1+cpu. CUDA was unavailable; all experiments ran on CPU. The study uses one principal seed and no large hyperparameter search.

Corpus-specific design choices

WikiText-2 contains section headings and special punctuation forms such as @-@, @.@ and @,@. Preserving them avoids silently changing the original token stream but means generated text can contain formatting artifacts. The 10,000-word cutoff was selected to keep the output vocabulary computationally manageable on CPU; it also forces uncommon proper nouns and inflections to share a single unknown token. These tradeoffs are central to interpreting both perplexity and downstream transfer.

2. Part I — predictive performance and cost

Model	Test CE	Test PPL	Training time	Stored size
Interpolated trigram	5.0650	158.38	1.2784 s counting	~320.13 MB Python counts
Vanilla RNN	4.8550	128.39	23.56 min	2.319M params / 8.85 MiB
LSTM	4.8790	131.49	24.74 min	2.408M params / 9.18 MiB
Causal Transformer	4.7389	114.31	31.75 min	2.198M params / 8.38 MiB

Table 1. Held-out next-token scores. Neural memory is FP32 parameter storage only; trigram memory is an approximate Python-object estimate, so these size numbers are not directly equivalent.

Prediction-quality ranking

The Transformer obtained the lowest test perplexity, 114.31, compared with 128.39 for the RNN, 131.49 for the LSTM and 158.38 for the aligned trigram. Relative to the RNN and LSTM, the Transformer perplexity is 11.0% and 13.1% lower. This is evidence that the trained Transformer assigned greater probability to held-out next tokens under the chosen setup. It is not evidence that attention alone caused the gain: the models differ in parameterization, normalization, dropout and optimization dynamics.

The RNN unexpectedly outperformed the LSTM by 3.10 perplexity points. Both recurrent models used the same 32-token horizon and five-epoch budget; the LSTM has more recurrent parameters and additional gates. A possible explanation is that these gates were not advantageous under the short horizon or training budget. A longer context or tuned schedule might reverse the ranking; that remains untested.

Teacher-forced inference throughput

Model	Median elapsed	Target positions/s	Timed operation
Trigram	0.0170 s	589,801	One target probability
RNN	0.1980 s	50,584	Full 10k-vocabulary logits
LSTM	0.1414 s	70,843	Full 10k-vocabulary logits
Transformer	0.1201 s	83,416	Full 10k-vocabulary logits

Table 2. Same 10,016 test target positions; neural batch size 32 and median of three post-warm-up runs, trigram median of five. The operations differ: the trigram evaluates a known target while neural models emit the complete vocabulary distribution.

Among the neural models, the Transformer achieved the highest batched throughput (83.4k positions/s), about 65% above the RNN and 18% above the LSTM. This is consistent with parallel processing across sequence positions, but optimized kernels and framework implementation can also contribute. In contrast, its 31.75-minute training run was the slowest, roughly 35% longer than the RNN. The results illustrate that training cost and batched inference cost need not rank models in the same order.

Cost interpretation

The trigram is extremely inexpensive to fit (1.28 s for counting) and its target-probability lookups are fast, but its Python count tables are memory-heavy and prediction quality is weaker. Neural models compress statistical regularities into a few million weights, but require iterative gradient updates. The Transformer in this experiment combines the smallest neural parameter count with the strongest held-out prediction score, showing that parameter count alone does not explain quality.

Interpreting the trigram baseline

Interpolation reduces the zero-probability problem that arises when an exact three-token sequence is absent from training. Validation selection favored weights 0.2/0.4/0.4 rather than the initial 0.1/0.3/0.6: validation perplexity fell from 185.77 to 169.38. The aligned test score was 158.38, compared with 157.77 under the earlier cross-boundary protocol. This small shift illustrates why the definition of a target and its available context must be documented before comparing model families.

3. Part I — convergence and generation behavior

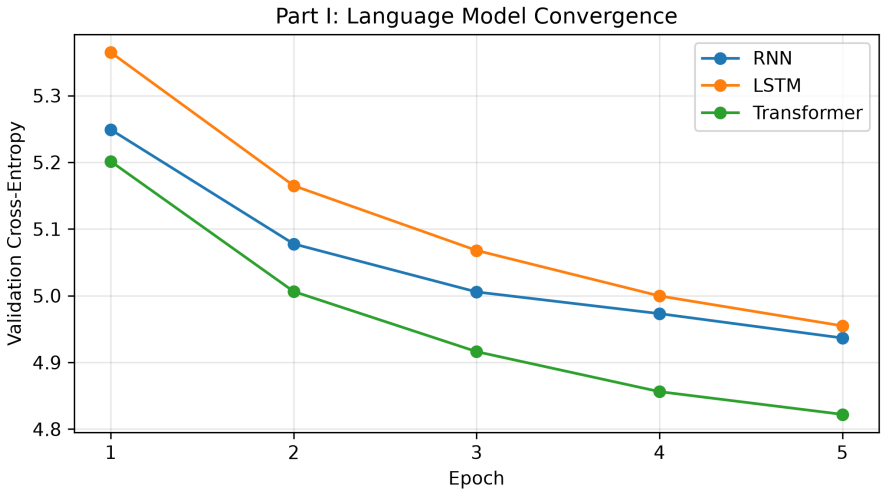


Figure 1. Validation cross-entropy for the RNN, LSTM and causal Transformer over five epochs.

Learning behavior

At epoch five, train/validation cross-entropies were 4.7449/4.9366 (RNN), 4.8684/4.9548 (LSTM) and 4.6852/4.8221 (Transformer). The respective gaps are 0.1917, 0.0864 and 0.1369. These gaps show that fitting the training stream is easier than generalizing to validation text, but a gap by itself does not prove that training has entered a harmful overfitting regime. The recorded best checkpoint for all three models was epoch five; additional epochs could have improved or worsened validation scores.

The Transformer finished with the lowest training and validation loss despite fewer parameters than either recurrent model. The LSTM has a smaller train-validation gap than the RNN, yet its validation loss is slightly higher; therefore the smallest generalization gap is not automatically the best model. Convergence should be judged using both the absolute validation loss and its trajectory.

Matched prompt continuations

Model	Excerpt from continuation of “the game”
Trigram	“the game ’s , who had <unk> into a small <unk> …”
RNN	“the game , where he was unable to gain an important role …”
LSTM	“the game , … = = = = = <unk> and career = = = …”
Transformer	“the game , where he was <unk> to get away …”

Table 3. All generated 30 new tokens with temperature 1.0, top-k 20 and seed 42; the same additional prompts “in the” and “the history of” were used for each model.

The trigram often joins locally frequent fragments without maintaining a topic, consistent with its two-token history. The RNN sometimes produces more fluent clauses, such as “unable to gain an important role,” yet its text still shifts topic. The LSTM reproduces sequences of equals signs used as Wikipedia heading markers. The Transformer also repeats high-probability expressions (“first season of the season”) and drifts from history to game reviews. Across models, <unk> visibly degrades readability.

What perplexity does not measure

Perplexity scores the probability of actual next tokens under teacher forcing, not the global coherence of freely sampled text. A model may improve average next-token probabilities while still producing contradictory, repetitive or semantically drifting passages. The three prompts provide qualitative illustrations only; they are not a statistically powered human evaluation. Fixed random seeds improve reproducibility within a model but do not force equivalent samples across architectures.

4. Part II — word-embedding ablation

The embedding experiment keeps the same one-layer LSTM, 100-dimensional inputs, 128-dimensional hidden state, 32-token chunks, Adam optimizer, data splits and five-epoch schedule. The variable is the input embedding strategy: random trainable (A), Skip-gram trained on the WikiText-2 training split and frozen (B), or public GloVe 6B 100d vectors and frozen (C). For frozen variants the entire 1,000,000-parameter embedding table was verified unchanged; their remaining 1,407,760 parameters were trained from scratch.

Variant	Val loss	Val PPL	Test loss	Test PPL	LM time
Random / trainable	4.9548	141.85	4.8790	131.49	24.74 min
Skip-gram / frozen	4.9220	137.28	4.8361	125.97	23.50 min
GloVe / frozen	4.9202	137.03	4.8445	127.03	25.44 min

Table 4. Embedding ablation: same LSTM architecture and training budget. Skip-gram is best on held-out test perplexity; GloVe is narrowly best on validation.

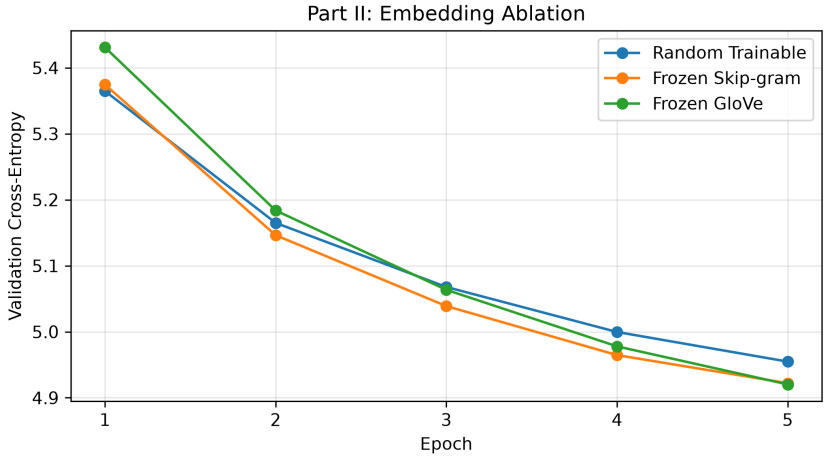


Figure 2. Validation-loss trajectories for three input-embedding settings.

What the numbers support

Frozen Skip-gram reduces test perplexity from 131.49 to 125.97 (4.2%), and frozen GloVe reduces it to 127.03 (3.4%). Skip-gram beats GloVe by only 1.06 perplexity points on test, while GloVe beats Skip-gram by 0.25 points on validation. The reversal between validation and test rankings and the small margins argue against a strong claim that either pretrained method is universally superior. All comparisons are based on one main seed.

Pretraining cost, coverage and confounding

Skip-gram used negative sampling (five negatives), window five, five epochs and 100-dimensional vectors; training the vectors separately took 41.76 seconds. It learned vectors for 9,999 non-padding vocabulary entries. Public GloVe covered 9,911 of 9,998 non-special tokens (99.13%); 87 missing tokens and <unk> used seeded Normal(0, 0.1) initialization, while <pad> was zero. The Skip-gram total pipeline therefore adds a small but explicit pretraining cost beyond its 23.50-minute LM training.

The trainable-vs-frozen comparison changes both embedding source and trainability. The Skip-gram-vs-GloVe comparison holds freezing fixed, but changes the embedding algorithm and available pretraining text. Accordingly, neither comparison isolates the causal effect of pretraining corpus size. The fixed embeddings may provide useful distributional structure early in LM training, but cannot adapt their token vectors to the next-token objective.

Representation quality versus predictive utility

Skip-gram nearest neighbors for “game” included “games” (cosine 0.746), “franchise” (0.670), “gameplay” (0.663) and “multiplayer” (0.644), indicating plausible distributional associations. Yet nearest-neighbor quality is not the same as language-model performance: the downstream LSTM must still learn how to combine fixed vectors over time. The small validation advantage for GloVe does not persist on test, emphasizing the need to compare embeddings by task metrics rather than illustrative neighbors alone.

5. Part II generation and Part III representation analysis

Embedding-variant generation

Under identical prompts and top-k decoding, all three LSTM embedding variants produced recognizable local phrases but struggled with discourse coherence. The trainable baseline emitted long runs of WikiText heading symbols; the frozen Skip-gram variant produced locally plausible fragments such as “where he was first to get at the end of ...” and also repeated heading markup. The visible GloVe samples similarly mixed unrelated subjects and included <unk>. Thus lower perplexity for the frozen variants did not yield an unambiguous qualitative improvement in the small prompt sample.

Downstream task and feature construction

Part III tests whether next-token representations transfer to binary IMDb sentiment classification. From the official IMDb training split, 4,000 balanced reviews were used for classifier training and 1,000 for validation; 1,000 balanced reviews from the official test split were held out. Both representation methods use the same examples and labels. The original Part I LSTM and Transformer were frozen; no language-model fine-tuning was performed.

IMDb HTML break tags were removed, text was lowercased and whitespace-tokenized with the WikiText vocabulary. Each review was truncated to at most 256 tokens and divided into independent chunks of at most 32. Hidden token states were mean-pooled across all retained tokens, giving one 128-dimensional LSTM vector or 100-dimensional Transformer vector per review. Separate StandardScaler + logistic regression classifiers (C=1.0, max_iter=1000, seed 42) used the same training/evaluation procedure.

Representation	Validation accuracy	Test accuracy	Test macro-F1
LSTM	68.6%	67.2%	0.6720
Transformer	68.7%	68.1%	0.6810

Table 5. The Transformer leads by 0.1 percentage points on validation and 0.9 points on test. Accuracy and macro-F1 agree because the class-balanced data produce similar positive/negative error rates.

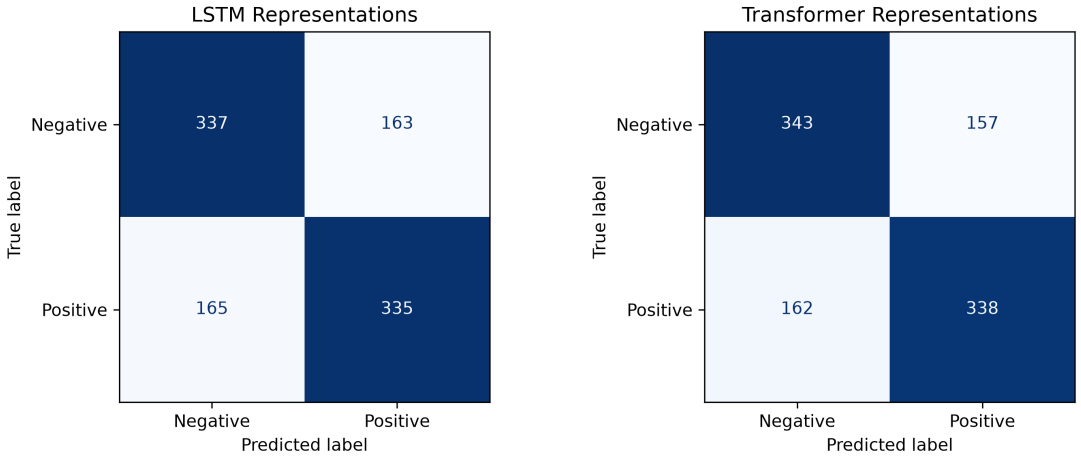


Figure 3. Held-out confusion matrices. LSTM: TN=337, FP=163, FN=165, TP=335. Transformer: TN=343, FP=157, FN=162, TP=338.

Linking language modeling and classification

The Transformer has 13.1% lower LM test perplexity than the LSTM, but only 0.9 percentage points higher sentiment accuracy. Their ranking agrees, yet the scale of the improvement differs substantially. Next-token prediction rewards broad distributional fit; sentiment classification depends on whether a pooled representation retains the polarity-bearing cues of an entire review. Different hidden dimensions, the classifier and the pooling rule also affect the downstream comparison.

The confusion matrices show a similar balance of positive and negative errors. The LSTM correctly identifies 337 negative and 335 positive reviews; the Transformer correctly identifies 343 negative and 338 positive reviews. Thus the Transformer’s advantage is not driven by a single class. However, a nine-review difference in a 1,000-review test sample is modest; no confidence interval or paired significance test was computed.

6. Error analysis, limitations and conclusions

Shared and complementary classification errors

Outcome across 1,000 held-out reviews	Count	Share
Both models correct	560	56.0%
Both models wrong	207	20.7%
Only LSTM correct	112	11.2%
Only Transformer correct	121	12.1%

The models differ in correctness on 233 reviews (23.3%). The Transformer correctly classifies nine more reviews, exactly matching the 0.9-point test accuracy gap. Although 207 errors are shared, the 112/121 split also shows that the models do not fail on identical examples. This suggests potentially complementary information, but no ensemble was evaluated.

Representative failures

Negative review, both predicted positive (test index 9). The reviewer strongly condemns a remake and several performances while praising two actors (“At the top of their game …”). A mean-pooled representation may give undue weight to the positive phrase and fail to capture the negative overall judgment. Positive review, both predicted negative (index 10). A concession about an unresolved ending is followed by praise for “hilarious one-liners” and “quirky characters” that “are sure to satisfy.” The models may underweight the concluding positive judgment. These explanations are hypotheses suggested by the text, not verified causal mechanisms.

Main limitations and threats to interpretation

Vocabulary and domain shift. The WikiText vocabulary collapses uncommon words into <unk>; one 113-token IMDb example had 23.89% unknowns versus 10.64% on the WikiText test corpus. That IMDb number is an example, not a corpus-wide rate. Whitespace tokenization also treats punctuation-attached words differently across domains. Context truncation. Independent 32-token LM chunks and a 256-token review cap can remove long-distance relationships and late sentiment cues; mean pooling further discards word order across token states.

Experimental control. The Transformer and recurrent models are not parameter-matched; Transformer dropout is 0.1 while the one-layer recurrent models use no dropout. One main seed and five epochs do not establish statistical significance or fully tuned convergence. The trigram uses a special first-position backoff mixture, and its target-probability lookup throughput is not equivalent to full-vocabulary neural logits. CPU kernel differences may explain part of the throughput ranking.

Integrated interpretation

Part I shows that the causal Transformer can outperform the other models in next-token likelihood under this particular CPU training budget, while being the fastest of the neural models at batched teacher-forced inference. Part II shows that fixed embeddings can outperform embeddings learned jointly with an LSTM, but the differences between Skip-gram and GloVe are small and their training sources differ. Part III shows that the Transformer’s larger language-modeling advantage corresponds to only a small downstream sentiment advantage. Together, these results argue for evaluating language models on the actual target task rather than selecting them solely by perplexity.

Lessons learned and possible extensions

The most useful next checks would be repeated seeds with uncertainty intervals; longer or overlapping contexts; better matched parameter budgets; a tokenization strategy that reduces IMDb out-of-vocabulary rates; and alternative pooling (e.g., final-state versus mean). Such experiments could distinguish optimization effects from architecture effects and determine whether downstream performance is limited primarily by representation quality, tokenization or review truncation. They were not run and are not included in the reported results.

Attribution and reproducibility

All reported scores and timings come from locally executed notebooks. External resources include WikiText-2, IMDb, GloVe and standard Python libraries. ChatGPT assisted with implementation guidance, debugging, experiment organization and drafting. The submitted README, requirements.txt, configurations, random seed and saved outputs document how to reproduce the main experiments.

Appendix A — Supplementary reproducibility details

A1. Software, environment and artifacts

Windows / Qualcomm Snapdragon X X1-26100 (ARM64), eight cores and eight logical processors, 2.96 GHz base clock; 16 GB installed RAM. Python 3.12.0; PyTorch 2.14.1+cpu; CPU-only execution. Other exact dependency versions are listed in requirements.txt. Models and feature artifacts were saved separately to avoid retraining for evaluation.

A2. Supplementary training and generation notes

The main language models each used five epochs; the best saved validation checkpoint was epoch five for the RNN, LSTM and Transformer. The trigram smoothing and interpolation settings were selected using validation data only. The neural training curves were saved individually and the combined comparison figures summarize the validation trajectories. Generation used 30 new tokens, temperature 1.0, top-k 20 and seed 42 for prompts “the game,” “in the” and “the history of.”

A3. Artifact inventory

Saved checkpoints include ngram_model.pkl, rnn_best.pt, lstm_best.pt, transformer_best.pt, lstm_skipgram_best.pt and lstm_glove_best.pt. Embedding matrices, JSON metric histories, generated-text examples, IMDb feature matrices and confusion-matrix images are stored in models/ and results/. The notebook execution order and data access instructions are described in README.md.

A4. References

[1] Merity et al. (2017), “Pointer Sentinel Mixture Models” (WikiText). [2] Maas et al. (2011), “Learning Word Vectors for Sentiment Analysis” (IMDb). [3] Pennington, Socher & Manning (2014), “GloVe: Global Vectors for Word Representation.” [4] Mikolov et al. (2013), “Distributed Representations of Words and Phrases and their Compositionality.” [5] PyTorch, Gensim, Hugging Face Datasets and scikit-learn documentation.